

AES128 ALGORITHM DOCUMENTATION

1. Introduction

The Advanced Encryption Standard (AES) is a symmetric block cipher algorithm used for secure encryption and

AES is extensively used in:

- * banking systems
- * wireless communication
- * cloud computing
- * embedded systems
- * military communication
- * cybersecurity infrastructure
- * secure internet communication

AES is called a:

```
```text id="1"
symmetric encryption algorithm
```
```

because:

```
```text id="2"
the same secret key is used for both encryption and decryption
```
```

AES provides:

- * confidentiality
- * data security
- * resistance against cryptographic attacks
- * high-speed hardware implementation capability

2. AES Variants

AES supports three different key sizes.

| AES Variant | Key Size | Number of Rounds |
|-------------|----------|------------------|
| AES-128 | 128-bit | 10 |
| AES-192 | 192-bit | 12 |
| AES-256 | 256-bit | 14 |

This project implements:

```
```text id="3"
AES-128
```
```

which uses:

- * 128-bit plaintext
- * 128-bit secret key
- * 10 encryption rounds

3. AES Block Structure

AES operates on:

```
```text id="4"
128-bit data blocks
```
```

The plaintext is divided into:

```
```text id="5"
16 bytes
```
```

These bytes are arranged into a:

```
```text id="6"
4 × 4 State Matrix
```
```

Example:

| Byte0 | Byte4 | Byte8 | Byte12 |
|-------|-------|-------|--------|
| ----- | ----- | ----- | ----- |

| | | | |
|-------|-------|--------|--------|
| Byte1 | Byte5 | Byte9 | Byte13 |
| Byte2 | Byte6 | Byte10 | Byte14 |
| Byte3 | Byte7 | Byte11 | Byte15 |

This matrix is called:

```
```text id="7"
State Matrix
```
```

All AES transformations operate on this State Matrix.

4. AES Encryption Overview

AES encryption converts:

```
```text id="8"
Plaintext → Ciphertext
```
```

using:

```
```text id="9"
Secret Key
```
```

AES encryption consists of:

- * Initial AddRoundKey
- * 9 Main Rounds
- * Final Round

5. AES Encryption Flow

```
```text id="10"
Plaintext
↓
Initial AddRoundKey
↓
SubBytes
↓
ShiftRows
↓
MixColumns
↓
AddRoundKey
↓
(Repeated for 9 rounds)
↓
Final Round
(SubBytes + ShiftRows + AddRoundKey)
↓
Ciphertext
```
```

6. Initial AddRoundKey

The first AES operation is:

```
```text id="11"
AddRoundKey
```
```

Purpose

AddRoundKey introduces:

- * key dependency
- * randomness
- * cryptographic security

Without the secret key:

```
```text id="12"
correct encryption and decryption are impossible
```
```

Operation

The plaintext is XORed with the secret key.

Equation:

```
```text id="13"
State = Plaintext XOR Key
```
```

Example:

```
```text id="14"
10101010
XOR
11001100
=
01100110
```
```

Hardware Perspective

In FPGA implementation:

```
```text id="15"
AddRoundKey is implemented using XOR gates
```
```

Advantages:

- * low hardware complexity
- * fast operation
- * reversible computation

7. SubBytes Transformation

After AddRoundKey:

```
```text id="16"
SubBytes
```
```

is performed.

Purpose of SubBytes

SubBytes provides:

```
```text id="17"
non-linearity
```
```

which protects AES against:

- * linear cryptanalysis
- * differential cryptanalysis

Working Principle

Each byte in the State Matrix is replaced using:

```
```text id="18"
S-Box Lookup Table
```
```

Example:

```
```text id="19"
Input Byte = 0x53
Output Byte = 0xED
```
```

S-Box

The AES S-Box is:

```

* predefined
* fixed
* nonlinear
* mathematically generated

```

The substitution is:

```

```text id="20"
one-to-one mapping
```

```

Hardware Implementation

In FPGA:

```

```text id="21"
S-Box is implemented using memory lookup
```

```

using:

```

```verilog id="pfx94s"
$readmemh("sbox.mem", sbox_mem);
```

```

Each input byte acts as:

```

```text id="22"
memory address
```

```

The corresponding memory value becomes:

```

```text id="23"
substituted output byte
```

```

8. ShiftRows Transformation

After SubBytes:

```

```text id="24"
ShiftRows
```

```

is performed.

Purpose

ShiftRows provides:

```

```text id="25"
diffusion
```

```

by redistributing bytes across columns.

This ensures:

```

```text id="26"
small plaintext changes affect multiple ciphertext bytes
```

```

Operation

Each row of the State Matrix is cyclically shifted.

| Row | Shift |
|-------|-----------------|
| Row 0 | No Shift |
| Row 1 | Left Shift by 1 |
| Row 2 | Left Shift by 2 |
| Row 3 | Left Shift by 3 |

Example

Before ShiftRows:

| | | | | | | | | |
|--|---|--|---|--|---|--|---|--|
| | A | | B | | C | | D | |
| | - | | - | | - | | - | |
| | E | | F | | G | | H | |
| | I | | J | | K | | L | |
| | M | | N | | O | | P | |

After ShiftRows:

| | | | | | | | | |
|--|---|--|---|--|---|--|---|--|
| | A | | B | | C | | D | |
| | - | | - | | - | | - | |
| | F | | G | | H | | E | |
| | K | | L | | I | | J | |
| | P | | M | | N | | O | |

Hardware Implementation

ShiftRows is hardware efficient because:

```
```text id="27"
it only rearranges wiring connections
```
```

No arithmetic operations are required.

9. MixColumns Transformation

After ShiftRows:

```
```text id="28"
MixColumns
```
```

is performed.

Purpose

MixColumns provides:

```
```text id="29"
strong diffusion
```
```

A small change in plaintext creates:

```
```text id="30"
large ciphertext variation
```
```

Working Principle

Each column of the State Matrix is multiplied using:

```
```text id="31"
Galois Field GF(2^8) arithmetic
```
```

AES uses:

```
```text id="32"
finite field mathematics
```
```

instead of normal multiplication.

AES MixColumns Matrix

```
```text id="33"
[02 03 01 01]
[01 02 03 01]
[01 01 02 03]
[03 01 01 02]
```
```

This matrix multiplies:

```
```text id="34"
State Column
```
```

to produce:

```
```text id="35"
mixed output column
```
```

Hardware Implementation

MixColumns uses:

- * xtime()
- * multiplication by 2
- * multiplication by 3

Example:

```
```verilog id="km4vps"
xtime(b)
```
```

performs:

```
```text id="36"
multiplication by 2 in GF(2^8)
```
```

Why MixColumns is Important

MixColumns ensures:

- * avalanche effect
- * diffusion
- * stronger security
- * ciphertext unpredictability

10. AddRoundKey

After MixColumns:

```
```text id="37"
AddRoundKey
```
```

is performed again.

Equation:

```
```text id="38"
State = State XOR RoundKey
```
```

This completes:

```
```text id="39"
one AES round
```
```

11. AES Final Encryption Round

The final AES round performs:

- * SubBytes
- * ShiftRows
- * AddRoundKey

BUT:

```
```text id="40"
MixColumns is NOT performed
```
```

The final output becomes:

```
```text id="41"
```

Ciphertext  
```\n

12. AES Decryption Overview

AES decryption reverses:

```\n`text id="42"\nencryption operations\n```\n

Operation:

```\n`text id="43"\nCiphertext → Plaintext\n```\n

using:

```\n`text id="44"\nsame secret key\n```\n

---

## # 13. AES Decryption Flow

```\n`text id="45"\nCiphertext\n↓\nAddRoundKey\n↓\nInvShiftRows\n↓\nInvSubBytes\n↓\nInvMixColumns\n↓\n(Repeated for all rounds)\n↓\nRecovered Plaintext\n```\n

14. InvShiftRows

InvShiftRows reverses:

```\n`text id="46"\nShiftRows\n```\n

---

### ## Operation

Rows are shifted:

```\n`text id="47"\nto the RIGHT\n```\n

| Row | Shift |
|-------|------------------|
| Row 0 | No Shift |
| Row 1 | Right Shift by 1 |
| Row 2 | Right Shift by 2 |
| Row 3 | Right Shift by 3 |

This restores:

```\n`text id="48"\noriginal byte positions\n```\n

---

## # 15. InvSubBytes

InvSubBytes reverses:

```\n`text id="49"\n

```

SubBytes
```

using:
```text id="50"
Inverse S-Box
```

Working Principle

Each substituted byte is converted back into:
```text id="51"
original byte
```

using:
```text id="52"
inv_sbox.mem
```

Hardware Implementation

InvSubBytes also uses:
```text id="53"
memory lookup table implementation
```

similar to encryption S-Box.

16. InvMixColumns

InvMixColumns reverses:
```text id="54"
MixColumns
```

using inverse Galois Field multiplication.

AES Inverse Matrix

```text id="55"
[14 11 13 09]
[09 14 11 13]
[13 09 14 11]
[11 13 09 14]
```

Operations Used

AES inverse multiplication uses:

- multiplication by 9
- multiplication by 11
- multiplication by 13
- multiplication by 14

These operations mathematically reverse:
```text id="56"
column mixing
```

performed during encryption.

17. AES Key Expansion

AES uses:
```text id="57"

```


Round Keys
```\n

generated from:

```\n`text id="58"  
original secret key
```\n

This process is called:

```\n`text id="59"  
Key Expansion
```\n

---

## ## Purpose

Key expansion ensures:

- \* different key usage in every round
- \* improved security
- \* resistance against attacks

---

## ## Key Expansion Components

AES key expansion uses:

- \* RotWord
- \* SubWord
- \* Round Constants (Rcon)
- \* XOR operations

---

## # 18. Importance of XOR in AES

AES heavily depends on:

```\n`text id="60"  
XOR operations
```\n

because XOR:

- \* is reversible
- \* is hardware efficient
- \* has low logic complexity
- \* provides strong key mixing

Important property:

```\n`text id="61"  
A XOR B XOR B = A
```\n

This enables:

```\n`text id="62"  
successful decryption
```\n

---

## # 19. AES Hardware Implementation on FPGA

AES is highly suitable for FPGA implementation because:

- \* operations are parallel
- \* transformations are modular
- \* lookup tables are FPGA-friendly
- \* XOR operations are hardware efficient

---

## ## FPGA Mapping

| AES Stage  | FPGA Implementation |
|------------|---------------------|
| SubBytes   | Memory Lookup       |
| ShiftRows  | Wiring Permutation  |
| MixColumns | Combinational Logic |

| AddRoundKey | XOR Network |

---

### ## Advantages of FPGA AES

FPGA implementation provides:

- \* parallel processing
- \* higher throughput
- \* low latency
- \* real-time execution
- \* hardware acceleration

---

### # 20. Vivado ILA Hardware Debugging

During FPGA implementation:

```text id="63"  
Vivado Integrated Logic Analyzer (ILA)
```

was used for:

- \* internal signal monitoring
- \* datapath debugging
- \* hardware verification

---

### ## Signals Monitored

The following signals were monitored:

- \* plaintext
- \* ciphertext
- \* SubBytes output
- \* ShiftRows output
- \* MixColumns output
- \* InvMixColumns output
- \* InvShiftRows output
- \* InvSubBytes output
- \* recovered plaintext

---

### ## Importance of ILA

ILA enabled:

- \* real-time FPGA debugging
- \* stage-by-stage verification
- \* datapath visualization
- \* hardware-level signal tracing

---

### # 21. Final Verification

The implemented AES datapath successfully demonstrated:

```text id="64"  
HELLO WORLD
↓
AES Encryption
↓
Ciphertext
↓
AES Decryption
↓
HELLO WORLD
```

This confirmed:

- \* successful encryption
- \* successful decryption
- \* correct inverse transformations
- \* proper FPGA integration
- \* hardware-level datapath verification

---

### # 22. Conclusion

AES is one of the most secure and widely used cryptographic algorithms in modern digital systems. Its modular

The FPGA-based AES128 implementation demonstrated:

- \* encryption logic
- \* decryption logic
- \* finite field arithmetic
- \* hardware datapath design
- \* FPGA debugging
- \* real-time verification

The successful implementation and hardware verification of AES128 on FPGA provides practical understanding

- \* cryptographic hardware acceleration
- \* RTL design
- \* FPGA implementation
- \* hardware verification methodologies
- \* digital hardware security systems